

CS 330, Fall 2026, Homework 2

Due Wednesday, September 16, 2026

For full credit, you must follow the guidelines laid out on the first page of homework 1.

1. **Colorful Path** The edges of an undirected graph G are each colored one of three colors: red, blue, or green. A path with 3 edges is called *colorful* if it consists of one red, one blue, and one green edge. The order of the colors is fixed to be red→blue→green. Design an algorithm that takes as input a graph G and returns all node quadruples that form a colorful path. The input is given as an adjacency list in the form of a nested hash table, where the value stored for an edge is its color. That is, $G[u][v] = c$ if and only if there is an edge from u to v with color c (where c is one of 'r', 'g', or 'b'). The output should be a list P of tuples of four nodes.

Analyze the time complexity of your algorithm in terms of the number of vertices n , the number of edges m , and the maximum degree D in the graph. For full credit your algorithm should run in time $O(n + mD^2)$, but slower correct algorithms (with reasonable complexity) will get nearly full credit.

2. **Part 1: Asymptotics**

For the following pairs of functions, clearly state whether $f(n) = O(g(n))$, $g(n) = O(f(n))$, both, or neither. Justify your answer for each claim.

- (a) $f(n) = n^{0.9}$ and $g(n) = \sqrt{2n}$
- (b) $f(n) = 2^n + n^2$ and $g(n) = n^6$
- (c) $f(n) = \log_2(n)$ and $g(n) = \log_2(n^2 + n)$
- (d) $f(n) = n^2 \log_2(n)$ and $g(n) = n^2 \cdot \sqrt{n}$

Hint1: Recall that if $f(n) = O(g(n))$ and $g(n) = O(f(n))$ then $f(n) = \Theta(g(n))$.

Hint2: Recall the combinatoric and analytic definitions of the asymptotic notations.

- $f(n) = O(g(n))$ means: there exist constants $c > 0$ and n_0 such that

$$\forall n \geq n_0, f(n) \leq c \cdot g(n).$$

- $f(n) = \Omega(g(n))$ means: there exist constants $c > 0$ and n_0 such that

$$\forall n \geq n_0, f(n) \geq c \cdot g(n).$$

- $f(n) = \Theta(g(n))$ means: there exist constants $c_1, c_2 > 0$ and n_0 such that

$$\forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Equivalently, in analytic terms:

- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$, then $f(n) = O(g(n))$.
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$, then $f(n) = \Omega(g(n))$.
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0$, then $f(n) = \Theta(g(n))$.

Hint3 (Growth-rate rule of thumb): When comparing asymptotic growth, you may freely use the hierarchy

$$\log n \ll n^c \ll a^n \quad \text{for any constant } c > 0 \text{ and any constant } a > 1$$

meaning that logarithms grow slower than any polynomial, and polynomials grow slower than any exponential.

Part 2: Runtime Analysis

For the following algorithms, analyze their running time as a function of the relevant input parameters. Clearly justify each bound.

- e) **Basic Merging.** You are given two sorted lists A and B of sizes a and b , respectively. Consider the standard merge procedure that repeatedly compares the smallest remaining elements of A and B and appends the smaller one to the output list.

Algorithm 2: Merge(A, B)

```

/*  $A$  and  $B$  are sorted lists */
1 Initialize empty list  $C$ ;
2 while  $A$  and  $B$  are both nonempty do
3   | Remove the smaller of the first elements of  $A$  and  $B$  and append it to  $C$ ;
4 Append any remaining elements of  $A$  or  $B$  to  $C$ ;
5 return  $C$ ;
```

Analyze the running time of this algorithm as a function of a and b .

- f) **Merging k Sorted Lists.** You are given k sorted lists $L_1, L_2, \dots, L_k$, each containing n elements. Using the Merge algorithm from part (e), we want to compute a single sorted list containing all kn elements.

Algorithm 3: MergeKSortedLists($L_1, L_2, \dots, L_k$)

```

/* Each list  $L_i$  contains  $n$  sorted elements */
1  $R \leftarrow L_1$ ;
2 for  $i \leftarrow 2$  to  $k$  do
3   |  $R \leftarrow \text{Merge}(R, L_i)$ ;
4 return  $R$ ;
```

Analyze the total running time of this algorithm as a function of n and k . You may use the result from part (e).

- g) **Counting Colors.** Similar to question 1, you are given a graph with n nodes and m edges that can be colored either red, blue, or green, and its adjacency list. We give an algorithm to compute the number of edges of each color.

Algorithm 4: ColorCounting(G)

```
1 blue-count  $\leftarrow$  0;  
2 red-count  $\leftarrow$  0;  
3 green-count  $\leftarrow$  0;  
4 for  $u$  in  $G$  do  
5   for  $v$  in  $G[u]$  do  
6     if  $G[u][v] == 'b'$  then  
7        $\text{blue-count} \leftarrow \text{blue-count} + 1$   
8     else if  $G[u][v] == 'g'$  then  
9        $\text{green-count} \leftarrow \text{green-count} + 1$   
10    else  
11       $\text{red-count} \leftarrow \text{red-count} + 1$   
12 return ( $\text{blue-count}/2, \text{green-count}/2, \text{red-count}/2$ )
```

Analyze the running time of this algorithm as a function of n and m .
